Chapter 9

Smart Contracts

Blockchain as a Programmable Platform

Teguh Indra Bayu, Ph.D.

FTI UKSW | TC789 Teknologi Blockchain

Source: Mastering Blockchain, 2nd Edition — Imran Bashir (Packt Publishing, 2018)

Learning Objectives

01

Trace the origins of smart contracts from Nick Szabo's 1990s theory to blockchain-era implementation

02

Define a smart contract and explain its four core properties

03

Explain why determinism is critical for smart contract execution on distributed blockchains

04

Distinguish between smart contracts and Ricardian contracts

05

Describe the Oracle problem and how Oracles provide external data to smart contracts

06

Analyze the DAO incident and its implications for smart contract security and testing

Chapter Overview

What This Chapter Covers

① **History**

From Nick Szabo's 1990s theory to Bitcoin's scripting

② **Definition**

Four core properties: executable, enforceable, semantic, secure

③ **Ricardian Contracts**

Bridging legal prose and machine-executable code

④ **Smart Contract Templates**

Domain-specific languages for financial instruments

⑤ **Oracles**

Connecting smart contracts to external real-world data

⑥ **Deployment & The DAO**

Platforms, languages, and real-world security lessons

History of Smart Contracts

Late 1990s

Nick Szabo coins the term "smart contract"

Defined as: "An electronic transaction protocol that executes the terms of a contract"

2009

Bitcoin launches with basic smart contract support

Limited scripting language; conditional logic for value transfer between parties

2015

Ethereum launches with full smart contract platform

Turing-complete EVM enables complex, programmable smart contracts at scale

2016

The DAO — a landmark (and a warning)

Largest crowdfunded project using smart contracts; exploited due to a code bug (~$50M)

What Is a Smart Contract?

> **"A smart contract is a secure and unstoppable computer program representing an agreement that is automatically executable and enforceable."**
>
> *— Mastering Blockchain, 2nd Ed. (Imran Bashir)*

Computer Program

Written in machine-understandable language; encodes agreements as business logic

Automatically Executable

Executes when predefined conditions are met — no human trigger required

Enforceable

All terms execute as defined, even in the presence of adversarial parties

"Code is Law"

No arbitrators, judges, or third-party intermediaries needed

Four Properties of a Smart Contract

The first two are minimum requirements. The last two are highly desirable.

1 **Automatically Executable** — **Required**

Executes when preset conditions are met. No manual trigger, no intermediary needed.

2 **Enforceable** — **Required**

All contract terms execute as programmed, even with adversarial parties present.

3 **Semantically Sound** — **Desirable**

Both humans and machines can understand the contract meaning — bridges legal prose and code.

4 **Secure & Unstoppable** — **Desirable**

Fault-tolerant and executable even under adverse external conditions.

Why Determinism Matters

Determinism: Same input → Same output, always, on every node in the network

Why It Is Essential

- Smart contracts run on every node in the network independently
- If results differ between nodes, consensus fails and the blockchain breaks
- Determinism guarantees every node reaches the exact same final state
- All smart contract languages must avoid non-deterministic operations

Common Non-Determinism Sources (To Avoid)

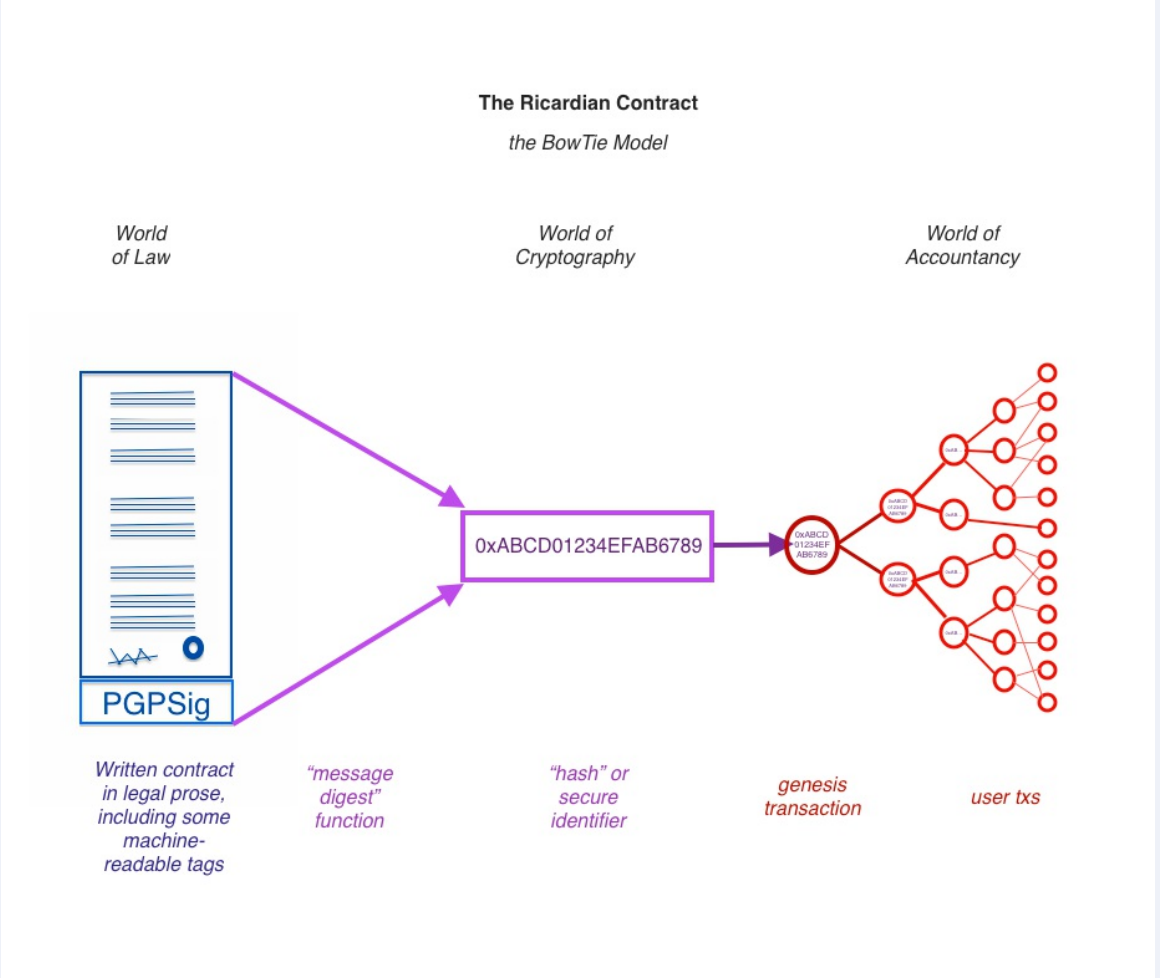
- Floating-point arithmetic (CPU/OS-dependent results)
- JavaScript Math functions (vary across browsers)
- Random number generation without shared seeds
- External timestamps (can differ slightly per node)

Ricardian Contracts

Concept

- Proposed by Ian Grigg (late 1990s) in 'Financial Cryptography in 7 Layers'
- Originally used in the Ricardo bond trading system
- Goal: A document acceptable by both courts of law and computer software
- Written contract + machine-readable tags in a single document
- Digitally signed by the issuer using their private key
- Hashed to produce a unique identifier used across all transactions

The Bowtie Model — Ricardian Contract



Note: Labels in diagram — World of Law (left), Hash identifier (center), World of Accountancy (right)

Smart Contract vs. Ricardian Contract

Comparing the two approaches — they are complementary, not competing.

Aspect	Smart Contract	Ricardian Contract
Primary Focus	Execution / Performance	Legal Definition / Semantics
Readable By	Machines (computers)	Humans AND machines
Legal Standing	"Code is Law" — self-enforcing	Legally binding document
Key Challenge	Determinism and security	Bridging legal prose and code
Example Use	Ethereum DApp / DeFi contract	Bond trading system (Ricardo)
Complementary?	*YES — an ideal smart legal contract combines BOTH dimensions*	

Ian Grigg's insight: Smart contracts are performance-rich; Ricardian contracts are semantics-rich. The ideal smart legal contract combines both.

Smart Contract Templates & Domain-Specific Languages

Templates standardize smart contracts for industries — especially finance.

Smart Contract Templates (Clack et al., 2016)

Standard reusable blueprints for financial instruments — derivatives, bonds, insurance

CLACK Language

A DSL for contract knowledge; supports legal prose AND multi-platform execution

Domain-Specific Languages (DSLs)

Limited expressiveness, optimized for one domain.
Different from general-purpose languages (GPLs).
Example: Solidity for Ethereum.

Smart Contract Languages

Solidity Ethereum (EVM)	Most widely used; C-like syntax
Vyper Ethereum (EVM)	Python-like; safer, more auditable
JavaScript Lisk	Familiar to web developers
Golang / Java / JS Hyperledger Fabric	Enterprise-grade chaincode

The Oracle Problem

Smart contracts cannot natively access external, real-world data.

Solution: Oracles — trusted interfaces that deliver external data to smart contracts.

Standard / Simple Oracle

Single trusted third-party data source.
Examples: weather agency, airport delay feed, financial market data provider.

Decentralized Oracle

Data aggregated from multiple independent sources; high agreement = trusted data. Reduces centralization risk.

Hardware Oracle

Real-world IoT sensor data with tamper-proof hardware and cryptographic integrity proofs.

Smart Oracle (Codius / Ripple)

An Oracle with added contract code execution capability; runs in a sandboxed environment (Google Native Client).

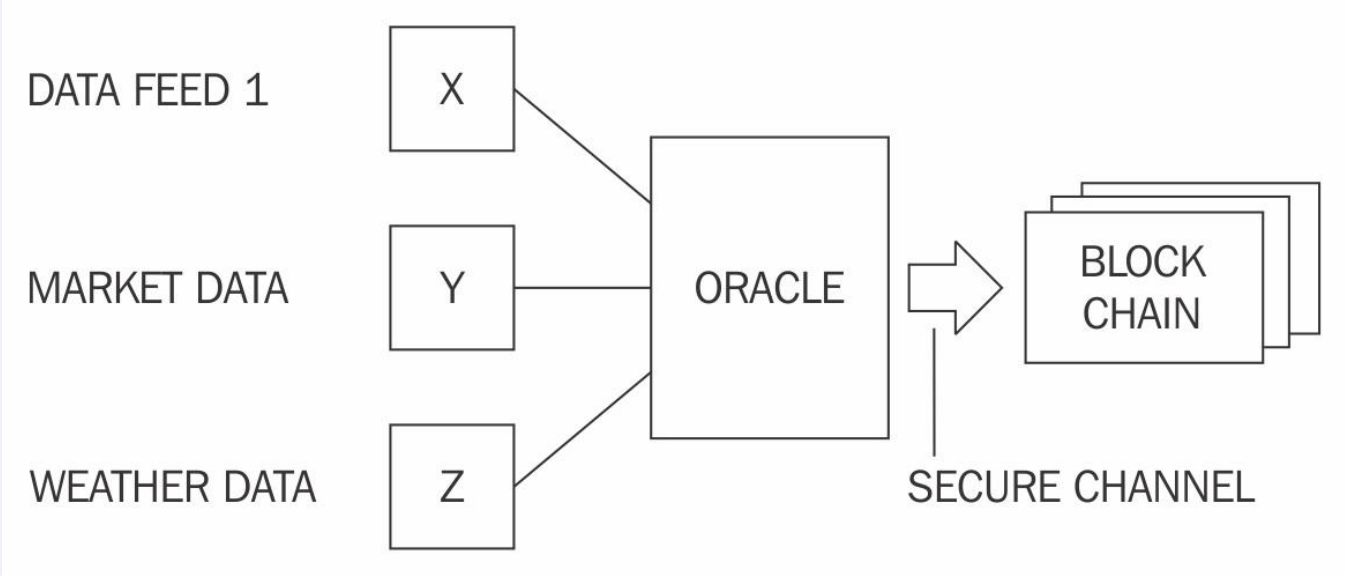
Oracle Architecture: How It Works

How Oracles Work

1. Smart contract subscribes to or queries an Oracle
2. Oracle retrieves data from an external source (API, IoT device, or web service)
3. Oracle authenticates data using TLSnotary — tamper-proof proof of data origin
4. Oracle pushes (or contract pulls) the verified data on-chain
5. Smart contract uses data to continue execution of business logic

Trust Issue: Oracles reintroduce a centralized point of trust — mitigated by decentralized oracle networks (e.g., Chainlink).

Oracle & Smart Contract Ecosystem (Mastering Blockchain, 2nd Ed.)



Deploying Smart Contracts on Blockchain

Key requirement for all platforms: Determinism — same result, everywhere, every time.

Ethereum — Primary Platform

- Natively designed for smart contract development
- Turing-complete Ethereum Virtual Machine (EVM)
- Smart contracts are part of broader DApps and DAOs
- Primary language: Solidity; also Vyper
- Most active developer ecosystem globally

Other Supporting Platforms

Bitcoin
Basic smart contracts via CLTV/nLocktime timelocks; scripting is limited, not Turing-complete

Hyperledger Fabric
Enterprise-grade chaincode; supports Golang, Java, JavaScript

Lisk
JavaScript smart contract platform for web developers

Corda / Monax
Focused on regulated financial markets and legal contracts

Stellar / Counterparty
Simpler smart contract templates for financial instruments

The DAO Incident (2016)

Apr 2016 — **LAUNCH**

DAO launches — raises ~$150M USD, the largest crowdfunded project at the time.

Jun 2016 — **EXPLOIT**

A re-entrancy bug is exploited. Approximately $50M USD is drained into a sub-account.

Jul 2016 — **HARD FORK**

Ethereum community votes for a hard fork to reverse the attack. The chain splits.

Aftermath — **RESULT**

ETH (new chain) vs. ETC (original — "code is law"). Ethereum Classic still runs today.

Lesson: Smart contracts execute exactly as coded — bugs cause irreversible financial harm. Formal verification and thorough testing are mandatory, not optional.

Key Terms & Definitions

Smart Contract

A secure, unstoppable program representing an agreement that is automatically executable and enforceable

Determinism

Same input always produces the same output on every node — essential for distributed consensus

Oracle

An interface that delivers external real-world data to smart contracts on the blockchain

Ricardian Contract

A legally binding document readable by humans AND computers; links legal prose to code via a cryptographic hash

Code is Law

The principle that smart contract code is the sole arbiter of execution — no judges or intermediaries needed

DSL (Domain-Specific Language)

A language with limited expressiveness, optimized for a specific domain. Example: Solidity for Ethereum.

DAO

Decentralized Autonomous Organization — governed entirely by smart contract code on a blockchain

TLSnotary

A mechanism providing cryptographic proof that Oracle data was retrieved from a specific source without tampering

Benefits vs. Limitations

Benefits

- Eliminates trusted intermediaries (lawyers, banks, escrow agents)
- Reduces transaction costs and settlement time
- Tamper-proof and transparent — code is publicly auditable
- Self-enforcing — executes automatically when conditions are met
- Applicable across industries: finance, healthcare, supply chain, government
- Global access — no geographic or institutional barriers

Limitations

- Cannot access external data without Oracles — reintroduces trust dependency
- Code bugs cause irreversible financial consequences — cannot be undone
- Not legally recognized as binding contracts in most jurisdictions
- Bridging natural language and code remains an unsolved challenge
- Determinism requirement restricts certain types of computation
- Not truly "smart" — only executes exactly what it is programmed to do

Key Takeaways

Chapter 9: Smart Contracts

1. Nick Szabo theorized smart contracts in the late 1990s; Bitcoin demonstrated them in 2009; Ethereum made them fully programmable in 2015.

2. A smart contract is automatically executable and enforceable — with determinism as its non-negotiable technical foundation.

3. Ricardian contracts bridge legal prose and code; an ideal smart legal contract combines execution and semantic richness.

4. Oracles are essential bridges connecting smart contracts to real-world data — but introduce centralization and trust challenges.

5. The DAO incident proved bugs in smart contract code can cause irreversible, large-scale financial harm. Formal verification is critical.

6. Smart contracts are not truly 'smart' — their correctness and safety depend entirely on the quality of the code.

Discussion & Q&A

Q1: Industry Impact

Which industries would benefit most from smart contracts — and which face the greatest adoption challenges? Why?

Q2: The Oracle Dilemma

If smart contracts depend on Oracles for external data, does this undermine the principle of decentralization? How might it be addressed?

Q3: Code Is Law — Or Is It?

The DAO's hard fork challenged 'code is law.' Was this the right decision? What are the implications for smart contract governance?

Q4: Legal Recognition

What technical and legal changes would be needed before smart contracts could fully replace traditional legal contracts?

Next: Chapter 10 — Ethereum 101 | The platform that made programmable smart contracts a reality